

ASSIGNMENT

Gradient Boosting & Ensemble Learning

Theory, Practical Implementation & Mini-Project

Course / Module	Machine Learning – Ensemble Methods
Topic	Gradient Boosting (GBM, XGBoost, LightGBM, CatBoost)
Assignment Type	Individual – Theory + Practical + Mini-Project
Duration	3 Days
Submission Mode	PDF Report + Jupyter Notebook (.ipynb) + GitHub link
Student Name	MAFEYISOPIN AYENI
Roll No. / ID	
Date of Submission	5/17/2026

1. Learning Outcomes

On successful completion of this assignment, the student will be able to:

- Explain the working principle of Gradient Boosting and how it differs from Bagging, Random Forests and AdaBoost.
- Identify the role of weak learners, residuals, learning rate and loss functions in boosting.
- Differentiate between GBM, XGBoost, LightGBM and CatBoost and justify when to use each.
- Implement Gradient Boosting classifiers and regressors using scikit-learn and XGBoost.
- Tune hyperparameters using cross-validation, grid search and early stopping.
- Evaluate models using appropriate metrics (Accuracy, Precision, Recall, F1, ROC-AUC).
- Apply SHAP / feature importance for model explainability.
- Convert a model into actionable business recommendations.

2. General Instructions

- Attempt ALL parts. The assignment is divided into five parts (A–E).
- Part A and Part B are theory; answer in your own words. Copy-pasted answers will not be accepted.
- Part C and Part D require coding. Submit a clean, well-commented Jupyter Notebook.
- Each plot must have a title, axis labels and a one-line interpretation below it.
- Use Python 3.10+ with scikit-learn, xgboost, lightgbm, pandas, matplotlib / seaborn and shap.
- Set `random_state = 42` wherever applicable to ensure reproducibility.
- Final deliverable: (a) PDF report, (b) Jupyter Notebook, (c) Public GitHub repository link.
- Late submissions will not be accepted; submit before the end of Day 3.
- Plagiarism check will be performed. Any copied work will result in the assignment being rejected.

3. Assignment Structure

Part	Focus
A	Conceptual Quick Answers – test your understanding of core ideas (5 short questions).
B	Theory & Short Answer – explain key concepts in depth (3 descriptive questions).
C	Practical Coding Exercises – build small models in Python (3 hands-on tasks).
D	Mini-Project – end-to-end Customer Churn Prediction using XGBoost.
E	Report, Documentation & Viva – communicate your work clearly.

4. Suggested 3-Day Plan

This is a recommended pace to help you finish on time. You may adjust based on your speed.

Day	Focus	What to complete
Day 1	Theory & Foundations	Read core concepts and complete Part A (5 conceptual questions) and Part B (3 theory questions). Set up Python environment with scikit-learn, xgboost and lightgbm.
Day 2	Practical Coding	Complete all three coding tasks in Part C (Titanic GBM, RF vs GBM comparison, XGBoost with early stopping). Save outputs and plots in your notebook.
Day 3	Mini-Project & Report	Complete the Customer Churn mini-project (Part D, all 6 steps). Write the 3–5 page PDF report (Part E) and push code to GitHub. Submit before end of day.

Part A – Conceptual Quick Answers

Answer each question in 2–3 sentences in your own words.

A1. Define Gradient Boosting in your own words.

Answer: Gradient Boosting is a machine learning method where many small “weak” models work together to make one strong model.

It works step-by-step:

- First model makes predictions
- Then the next model focuses on correcting the mistakes from the first one
- Another model corrects the remaining mistakes again
- This keeps happening until the predictions become much better

Think of it like a student correcting error after every test until they improve.

Usually, the small models used are decision trees.

A2. What is a weak learner, and why are shallow trees typically used?

Answer: A **weak learner** is a model that performs only slightly better than random guessing. In boosting, weak learners are combined together to create a strong model.

Example:

- A small decision tree with only a few splits

Why use shallow trees?

- Faster to train
 - Prevents overfitting
 - Focuses on simple patterns first
 - Works better in boosting because many simple trees combine together gradually
- Most boosting algorithms use shallow trees called **decision stumps** or trees with low depths.

A3. State two key differences between Bagging and Boosting.

Answer:

Bagging	Boosting
Models train independently	Models train one after another
Reduces variance	Reduces bias and errors
Example: Random Forest	Example: XGBoost

Explanation in another simple way:

- **Bagging** = many models vote together
- **Boosting** = each new model learns from previous mistakes

A4. What is the role of the learning rate (shrinkage) in Gradient Boosting?

Answer: The learning rate controls **how much each new tree contributes** to the final prediction.

A5. Name three boosting libraries and one unique feature of each.

Answer:

Library	Feature
XGBoost	Very fast and highly optimized
LightGBM	Handles large datasets efficiently
CatBoost	Works very well with categorical data automatically

These are some of the most popular boosting libraries used in AI and ML today.

Part B – Theory & Short Answer

Answer each question in approximately half a page. Use diagrams where helpful.

B1. Explain the step-by-step working of Gradient Boosting using a small numerical example. Show how residuals are computed and how the prediction is updated after each tree.

Answer:

Step-by-Step Working of Gradient Boosting

Gradient Boosting builds models sequentially, where each new tree tries to correct the errors made by the previous trees.

Suppose we have this simple dataset:

X	Actual y
1	10
2	12
3	14

Step 1: Start with Initial Prediction

The model first predicts the average of all target values.

$$\text{Mean} = \frac{10 + 12 + 14}{3} = 12$$

So, the first prediction for every row is:

X	Actual y	Prediction
1	10	12
2	12	12
3	14	12

Step 2: Compute Residuals (Errors)

Residual = Actual – Predicted

X	Actual y	Prediction	Residual
1	10	12	-2
2	12	12	0
3	14	12	+2

The next decision tree is trained on these residuals instead of the original values.

Step 3: Train the First Tree on Residuals

The tree learns:

- For X=1 → residual = -2
- For X=2 → residual = 0
- For X=3 → residual = +2

Suppose the learning rate is:

$$\eta = 0.5$$

Updated prediction formula:

$$\text{New Prediction} = \text{Old Prediction} + \eta \times \text{Tree Output}$$

Step 4: Update Predictions

X	Old Prediction	Tree Output	New Prediction
1	12	-2	$12 + 0.5(-2) = 11$
2	12	0	12
3	12	+2	13

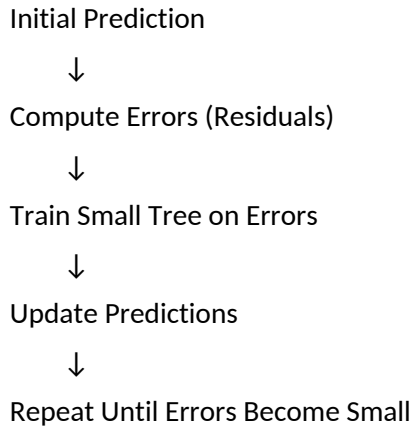
The predictions are now closer to the actual values.

Step 5: Repeat the Process

New residuals are computed again:

X	Actual y	Prediction	Residual
1	10	11	-1
2	12	12	0
3	14	13	+1

Another tree is trained to correct these remaining errors. This process continues until the model achieves good accuracy.

Simple Diagram

Gradient Boosting works because each new tree focuses on correcting previous mistakes.

- B2. Compare and contrast AdaBoost and Gradient Boosting. Discuss how each handles errors, weights, and loss functions.

Answer:

AdaBoost and Gradient Boosting are both boosting algorithms that combine weak learners to create a strong predictive model. However, they differ in how they handle errors, weights, and optimization.

1. Error Handling**AdaBoost**

AdaBoost focuses mainly on misclassified observations. After each model is trained, it increases the weights of incorrectly predicted samples, so the next learner pays more attention to them.

Example:

- Wrong predictions receive higher importance
 - Correct predictions receive lower importance
- This forces future models to focus on difficult cases.

Gradient Boosting

Gradient Boosting does not directly change observation weights. Instead, it minimizes prediction errors by fitting new trees to the residuals (errors) from previous predictions.

It learns using gradient descent optimization.

2. Weights**AdaBoost**

- Uses sample weights heavily
- Misclassified points get larger weights
- Final prediction is a weighted combination of weak learners

Gradient Boosting

- Does not mainly rely on changing sample weights
- Uses residual correction instead
- Trees are added sequentially to reduce total loss

3. Loss Functions

AdaBoost

Typically uses exponential loss:

$$L = e^{-yf(x)}$$

Mainly designed for classification tasks.

Gradient Boosting

Can use many loss functions depending on the problem:

- Mean Squared Error for regression
- Log Loss for classification
- Custom losses

This makes Gradient Boosting more flexible.

Simple Summary Table

Feature	AdaBoost	Gradient Boosting
Main Goal	Focus on wrong predictions	Reduce residual errors
Uses Weights?	Yes	Mostly no
Loss Function	Exponential loss	Multiple loss functions
Training Style	Reweights observations	Fits residuals
Common Use	Classification	Regression & Classification

- B3. Compare XGBoost, LightGBM and CatBoost across the following dimensions: tree-growth strategy, handling of categorical variables, speed, and typical use cases.

Answer:

XGBoost, LightGBM, and CatBoost are advanced Gradient Boosting libraries widely used in machine learning.

Feature	XGBoost	LightGBM	CatBoost
---------	---------	----------	----------

Tree Growth Strategy	Level-wise growth	Leaf-wise growth	Symmetric trees
Categorical Variables	Requires encoding	Limited native support	Native handling
Speed	Fast	Very fast	Moderate
Memory Usage	Moderate	Low	Moderate
Accuracy	High	High	High
Best Use Cases	General ML tasks	Large datasets	Many categorical features

1. Tree-Growth Strategy

XGBoost

Uses level-wise tree growth.

Level-wise:

```

      Root
     /  \
    A    B
  
```

It expands all nodes level by level. This creates balanced trees and helps avoid overfitting.

LightGBM

Uses leaf-wise growth.

Leaf-wise:

```

      Root
      /
     A
    /
   B
  
```

It grows the leaf with the highest error reduction first. This is faster and often more accurate but may overfit on small datasets.

CatBoost

Uses symmetric trees where splits happen evenly across all branches. This improves prediction speed and stability.

2. Handling Categorical Variables

XGBoost

Categorical variables must usually be converted using:

- One-hot encoding
- Label encoding

LightGBM

Provides limited native categorical support but may still require preprocessing.

CatBoost

Best for categorical data because it automatically processes categorical features without heavy preprocessing.

3. Speed

- **LightGBM** is usually the fastest for large datasets.
- **XGBoost** is highly optimized and balanced.
- **CatBoost** can be slightly slower because of extra categorical processing.

4. Typical Use Cases**XGBoost**

- Kaggle competitions
- General structured/tabular data
- Balanced performance and interpretability

LightGBM

- Very large datasets
- High-speed training
- Memory-efficient applications

CatBoost

- Datasets with many categorical variables
- Business/customer data
- Cases where preprocessing needs to be minimized

All three libraries are powerful and widely used in modern machine learning systems.

Part C – Practical Coding Exercises

Use Python and submit code, output and a brief interpretation for each task.

Task C1 – Baseline GBM on Titanic Dataset

- Load the Titanic dataset (from seaborn or Kaggle).
- Perform basic preprocessing (handle missing values, encode categorical features).
- Train a GradientBoostingClassifier and report Accuracy, Precision, Recall and F1.
- Compare briefly with a Logistic Regression baseline.

Task C2 – Random Forest vs Gradient Boosting

- On any classification dataset of your choice ($\geq 1,000$ rows), train both a Random Forest and a Gradient Boosting Classifier.
- Compare them on Accuracy, Training Time and ROC-AUC.
- Present results in a small table and write a short discussion of which performed better and why.

Task C3 – XGBoost with Early Stopping & Tuning

- Install xgboost and train an XGBClassifier on any classification dataset.
- Use a validation set with `early_stopping_rounds = 20`.
- Tune at least two hyperparameters (e.g. `learning_rate`, `max_depth`) and report the best configuration.
- Plot training vs validation loss; identify the optimal number of trees.

Part D – Mini-Project: Customer Churn Prediction using XGBoost

Deliverable: Jupyter Notebook + Report Section + GitHub repository

D.1 Business Problem

A telecom company is losing a significant number of customers every month. As a Data Scientist, you are asked to build a model that predicts the probability of churn for each customer so that the retention team can target high-risk customers with offers and improved service.

Recommended dataset: Telco Customer Churn (Kaggle) –
<https://www.kaggle.com/datasets/blastchar/telco-customer-churn>

D.2 Project Workflow

Step	Activity
1	Data cleaning & quick EDA – handle nulls, encode categorical variables, produce 2–3 key plots (e.g. churn distribution, churn vs contract type).
2	Feature engineering – create at least 1 new feature (e.g. tenure group or total services count).
3	Train two models on the same train/test split: GradientBoostingClassifier and XGBoost. Compare results.
4	Handle class imbalance using scale_pos_weight or class_weight; do a small hyperparameter tune on the better model.
5	Evaluate using Accuracy, Precision, Recall, F1 and ROC-AUC. Briefly explain why Recall matters in churn prediction.
6	Identify the top churn drivers (feature importance or SHAP) and give 2 concrete business recommendations.

D.3 Suggested Starter Code

```
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import classification_report, roc_auc_score
from xgboost import XGBClassifier
import shap

df = pd.read_csv('telco_churn.csv')
# ... preprocessing here ...
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
```

```
model = XGBClassifier(  
    n_estimators=500, learning_rate=0.05, max_depth=4,  
    scale_pos_weight=3, eval_metric='auc', random_state=42)  
model.fit(X_train, y_train,  
        eval_set=[(X_test, y_test)], early_stopping_rounds=20, verbose=False)  
  
print('Test ROC-AUC:', roc_auc_score(y_test, model.predict_proba(X_test)[:, 1]))
```

Part E – Report, Documentation & Viva

E.1 Report Structure & Quality

Your final PDF report (3–5 pages) should include the following sections:

- Title page (student details + project title).
- Abstract (150 words).
- Introduction & Business Problem.
- Dataset description and EDA highlights (with at least 3 charts).
- Methodology – preprocessing, feature engineering, model choice, tuning strategy.
- Results – comparison table of all models and final chosen model.
- Explainability – SHAP plots and interpretation.
- Business Recommendations.
- Conclusion & Future Work.
- References (papers, blogs, documentation).

E.2 Viva Voce

Be prepared for a 5–10 minute viva where you may be asked questions such as:

1. Why did you choose XGBoost over Logistic Regression?
2. Why is Recall more important than Precision in churn prediction?
3. How did you handle class imbalance, and what was the effect?
4. Which features turned out to be the strongest churn drivers? Were they expected?

5. Evaluation Criteria

Criterion	Excellent	Good	Average	Poor
Conceptual understanding	Clear, original explanations with examples	Mostly correct, some depth	Surface-level answers	Incorrect / copied
Code quality	Modular, commented, reproducible	Runs, mostly clean	Runs with issues	Doesn't run
Model performance	Well-tuned, justified choices	Reasonable results	Default settings only	Poor / no tuning
EDA & visualisation	Insightful, well-labelled plots	Clear plots, brief insight	Plots without insight	Missing
Explainability	SHAP + clear business	SHAP shown, basic reading	Feature importance	Missing

	reading		only	
Report & presentation	Professional, well-structured	Complete, minor gaps	Sketchy structure	Incomplete

6. Submission Checklist

- ☐ Final PDF report (3–5 pages).
- ☐ Jupyter notebook (.ipynb) with all code, outputs and markdown explanations.
- ☐ Public GitHub repository link added to the cover page.
- ☐ Dataset link or dataset file (if size permits).
- ☐ Self-declaration of originality signed and dated.

7. Suggested Resources

- Friedman, J. H. (2001) – "Greedy Function Approximation: A Gradient Boosting Machine".
- Chen, T. & Guestrin, C. (2016) – "XGBoost: A Scalable Tree Boosting System".
- scikit-learn documentation – Ensemble Methods.
- XGBoost official docs – <https://xgboost.readthedocs.io>
- LightGBM official docs – <https://lightgbm.readthedocs.io>
- CatBoost official docs – <https://catboost.ai>
- SHAP documentation – <https://shap.readthedocs.io>
- Kaggle Learn – Intermediate Machine Learning course.

— End of Assignment —